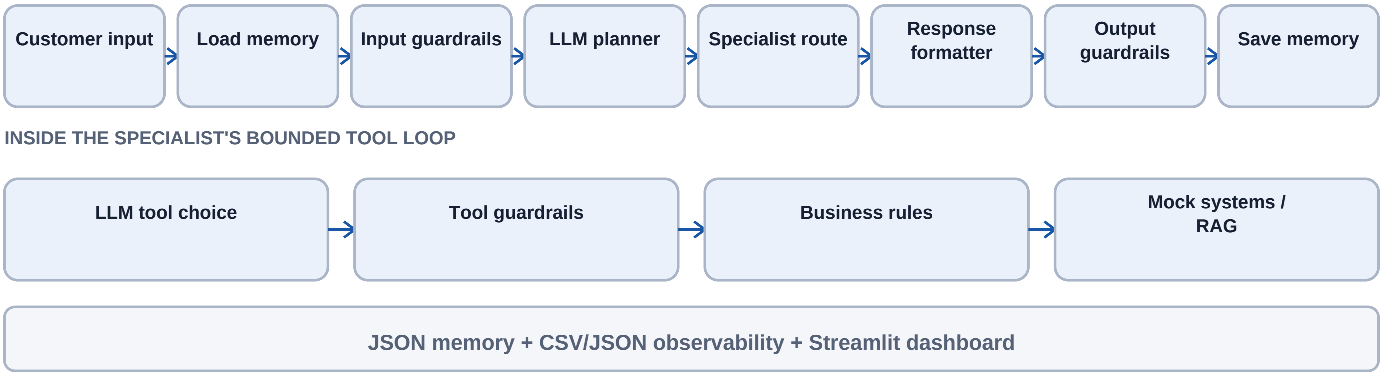
Customer Operations Agent

A simple view of architecture, controls, decisions, and tradeoffs

Architecture



Key design decisions and tradeoffs

- LangGraph was chosen instead of one large prompt: each step is visible, testable, and recoverable; the tradeoff is more orchestration code and model calls.
- OpenAI performs intent routing and genuine tool selection; deterministic Pydantic validation and the BusinessRuleEngine retain authority over execution.
- FAISS provides fast local semantic retrieval and JSON keeps memory simple; both are inexpensive for a demo but not suitable as shared production stores.
- Mock CRM, billing, and ticket systems make actions safe and repeatable while preserving realistic budgets, approvals, IDs, and audit traces.
- Policy and troubleshooting answers fail closed without grounding; this reduces hallucination risk but may increase safe escalations when retrieval fails.

Guardrail approach

- Before the LLM: prompt-injection detection, PII masking, abuse review, and blocked-input routing.
- Before each action: tool allowlist, strict arguments, customer isolation, maximum calls, dead-loop detection, and circuit breaking.
- Financial controls: refunds above INR 1,000 pause for trusted yes/no approval; offers are capped at 20% and share an INR 10,000 run budget.
- Before delivery: structured output schema, confidence threshold, citation requirement, PII masking, and safe fallback.

Evaluation, rollout, cost, and risk

Submission scope: Tier 1 and Tier 2; Tier 3 is not claimed

Evaluation results

- 45 deterministic tests pass: 12 integration test cases across 11 scenario categories, plus unit coverage for routing, tools, memory, guardrails, RAG, observability, and HITL.
- Tool-loop tests prove model-selected customer lookup, KB search, grounded citation capture, strict customer isolation, and controlled action execution.
- HITL tests prove both paths: approval revalidates and issues the refund; rejection leaves it unexecuted and creates a human escalation.
- Failure scenarios verify prompt-injection blocking, budget exhaustion, circuit breaking, dead-loop detection, and safe provider fallback.

Production rollout plan

- 1. Replace local mocks with sandbox adapters while retaining the same validated tool contracts.
- 2. Run in shadow mode: compare proposed actions with human decisions and block all writes.
- 3. Canary low-risk billing and troubleshooting requests; keep refunds and offers approval-only.
- 4. Add durable conversation, approval, and budget storage with idempotency keys and role-based access.
- 5. Add distributed tracing, alerting, secrets management, model/version pinning, and retrieval-quality monitoring.
- 6. Expand gradually only when safety, groundedness, latency, cost, and business KPI thresholds remain healthy.

Cost and risk notes

- Primary variable costs are planner/specialist tokens and KB embeddings; cache stable embeddings and keep prompts/tool results compact.
- Retention discounts directly reduce revenue, so eligibility and aggregate budget are enforced in code rather than prompts.
- Provider quota, latency, and outages can interrupt automation; retries, circuit breakers, observability, and human fallback limit impact.
- PII and billing data create privacy risk; mask before model use, minimize logs, encrypt durable stores, and apply retention policies.
- Model confidence is self-reported and not calibrated; production thresholds require empirical validation against human-reviewed outcomes.

Business KPIs

- Resolution rate and escalation rate by intent.
- Net retained monthly revenue minus discount cost.
- Cost per resolved request and p95 end-to-end latency.
- Grounding failures, approval violations, injection blocks, and incorrect tool calls.